T.C.
FIRAT ÜNİVERSİTESİ

STAJ DEFTERİ

MÜHENDİSLİK FAKÜLTESİ

"Çalışmak demek, boşuna yorulmak, terlemek değildir. Zamanın gereklerine göre bilim teknik ve her türlü uygar buluşlardan azami derecede istifade etmek zorunludur."

M. Atatürk

T.C.
FIRAT ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
STAJ DEFTERİ

ÖĞRENCİNİN	**BÖLÜMÜ**	Yazılım Mühendisliği			**FOTOĞRAF**
	NUMARASI	230290041			
	ADI VE SOYADI	Yaşar Buğra Erbay			
	STAJ DÖNEMİ	II. Dönem: 21 Ağustos 2025			
	STAJ BAŞ. TARİHİ	**11/08/2025**	**TOPLAM İŞ GÜNÜ SAYISI**	**20**	
	STAJ BİTİŞ TARİHİ	**05/09/2025**			

STAJ YAPILAN İŞ YERİ BİLGİLERİ

İŞ YERİ	ADI	Sia Teknoloji	
	ADRESİ	Fırat Teknokent Ek Bina, Z04, 23200 Elazığ Merkez/Elazığ	
	TELEFON-FAKS	532 170 74 28	
SORUMLU MÜHENDİS	ADI VE SOYADI	Safa Merve Taşçı	
	ÜNVANI	Yazılım Mühendisi	
	GÖREVİ	Stajyerin teknik takibi, görev planlama	
AMİR	ADI VE SOYADI	Safa Merve Taşçı	
	ÜNVANI	Yazılım Mühendisi	
	GÖREVİ	Stajyerin teknik takibi, görev planlama	

BÖLÜM STAJ KOMİSYONU DEĞERLENDİRME SONUCU

Yapılan pratik çalışmanın ……. iş günü ……. Dönem stajı olarak kabul edilmiştir /edilmemiştir.
Yapılan pratik çalışmanın ……. iş günü ……. Dönem stajı olarak kabul edilmiştir /edilmemiştir.

STAJ KOMİSYONU

……/……/……..	……/……/……..	……/……/……..
İMZA	İMZA	İMZA
BAŞKAN	ÜYE	ÜYE

İÇİNDEKİLER

KONU Sayfa No

Şekil, Çizelge ve Ekler Listesi ……...2
Stajın Yapıldığı Kurum veya Kuruluşun Tanımı……..3
Giriş……..……..4
Haftalık Çalışma Tabloları……..……5
Haftalık Çalışma Tabloları……..……6
Haftalık Çalışma Tabloları……..……7
Gün 1 ……..…….. 8
Gün 2 ……..…….. 9
Gün 3 ……..…… 10
Gün 4 ……..…… 11
Gün 5 ……..…… 12
Gün 6 ……..…… 13
Gün 7 ……..…… 14
Gün 8 ……..…… 15
Gün 9 ……..…… 16
Gün 9..……..…….. 17
Gün 10 ……..…….. 18
Gün 11 ……..…….. 19
Gün 12 ……..…….. 20
Gün 13 ……..…….. 21
Gün 14 ……..…….. 22
Gün 15 ……..…….. 23
Gün 15 ……..…….. 24
Gün 16 ……..…….. 25
Gün 17 ……..…….. 26
Gün 18 ……..…….. 27
Gün 19 ……..…….. 28
Gün 20 ……..…….. 29

ŞEKİL, ÇİZELGE VE EKLER LİSTESİ

Şekil, Çizelge veya Ek No Sayfa No

Şekil 1. Anel Plaza İş Merkezi Girişi …….. .3
Şekil 2. Proje İçin Githup Reposu…….... .8
Şekil 3. Proje İçin Githup Reposu ...…….. 9
Şekil 4. React Projesi Kaynak Kodlar……..10
Şekil 5. React Projesi Kaynak Kodlar ……11
Şekil 6. Redux Toolkit Slice Yapısı ve Durum (State) Yönetimi Kodları ……...12
Şekil 7. React Router ile Sayfa Yönlendirme Yapılandırması ……...13
Şekil 8. Firebase Bağlantı ve Yapılandırma Dosyası (firebase.js)…….....14
Şekil 9. Dosya Hiyerarşisi ve App.js Konfigürasyonu.....……..15
Şekil 10. Auth Bileşeni ve Firebase Giriş/Kayıt Kodları …...…….....16
Şekil 11. Kullanıcı Kayıt (Register) Ekranı Arayüzü ……..17
Şekil 12. MDB React Navbar Kodları ve Menü Görünümü ……18
Şekil 13. Redux Provider Entegrasyonu ve Spinner Bileşeni Kodları..……... 19
Şekil 14. Blog Gönderisi Ekleme ve Düzenleme Formu Arayüzü……... 20
Şekil 15. Firestore Veritabanı Görünümü ve Veri Çekme (API) Kodları……....21
Şekil 16. Home Sayfası ve Blog Listeleme Arayüzü……22
Şekil 17. Blog Güncelleme/Silme Fonksiyonları ve Detay Sayfası Kodları …….....24
Şekil 18. Kullanıcıya Özel Gönderileri Listeleyen MyPost Bileşeni Kodları ……25
Şekil 19. Oda Detayları ve Katkı (Contribute) Arayüzü ……26
Şekil 20. Yeni Oda Oluşturma (Create Room) Formu Arayüzü ……...28

STAJIN YAPILDIĞI KURUM VEYA KURULUŞUN TANIMI

Sia Teknoloji, **2013** yılında **TÜBİTAK BİGG** desteğiyle kurulmuş, iş dünyasına özel yazılım çözümleri geliştiren bir teknoloji şirketidir. Merkez adresi Fırat Teknokent olup başta web tabanlı kurumsal uygulamalar, süreç otomasyonu ve veri odaklı hizmetler olmak üzere geniş bir yazılım portföyüne sahiptir. Kuruluşundan bu yana çok sayıda özel sektör firması ve kamu kurumu ile çalışarak **50'ye yakın** projeyi başarıyla tamamlamış, ölçeklenebilir ve sürdürülebilir teknolojik ürünler üretme yetkinliğini kanıtlamıştır. Organizasyon yapısı; Yazılım Geliştirme, Analiz/Ürün Yönetimi, Kalite Güvencesi, Sistem/DevOps ve Destek birimlerinden oluşmakta; yönetim yapısı proje ve ürün odaklı çalışacak şekilde yalın, çevik ve hesap verebilir bir yönetişim yaklaşımı benimsemektedir. Çalışan profili; genç, öğrenmeye açık ve yeni teknolojileri hızla uygulamaya alan yazılım mühendisleri, iş analistleri ve test uzmanlarından oluşur. Şirket, müşteriye özel uygulama geliştirme, mevcut sistemlerin modernizasyonu, entegrasyon ve bakım-destek hizmetleri sunarken; danışmanlık ve eğitim gibi katma değerli servislerle kurumların dijital dönüşüm yolculuğunu hızlandırmayı hedefler. Çalışma ortamı **2** kişilik bir ekip yapısına sahip olup, günlük işleyiş **Scrum/Kanban** pratikleriyle sprint planlama, günlük durum toplantıları ve düzenli geribildirim döngüleri etrafında yürütülür; sürüm kontrolü Git/GitHub üzerinden yönetilir, kod inceleme ve birleştirme süreçleri standarttır. Staj kapsamında yer aldığım **Yazılım Geliştirme** bölümünde görevler; ön yüz geliştirme, hata düzeltme, küçük özelliklerin uygulanması ve kod gözden geçirme süreçlerine destek şeklindedir; kullanılan başlıca teknolojiler **React**, **Redux Toolkit/RTK Query**, **React Router**, **Firebase (Auth, Firestore, Storage)**, arayüz bileşenleri için **MDB UI Kit**, sürüm kontrolü için **Git/GitHub** ve geliştirme ortamı olarak **VS Code**'dur. Kuruluşta yürütülen projeler çoğunlukla müşteri süreçlerini dijitalleştiren web uygulamaları, raporlama panelleri ve entegrasyon çözümleridir; ayrıca bulut tabanlı mimari, performans ve güvenlik odaklı **Ar-Ge** çalışmaları şirketin öncelikli gündemleri arasındadır. Bu yapı sayesinde Sia Teknoloji, teknolojiyi iş dünyasına etkin şekilde entegre ederek müşterilerine sürdürülebilir değer katmayı ve sektörde öncü konumunu güçlendirmeyi hedeflemektedir

Şekil 1. Anel Plaza İş Merkezi Girişi

GİRİŞ

Bu stajın amacı, derslerde edinilen kuramsal bilgileri gerçek iş akışlarıyla birleştirerek profesyonel yazılım geliştirme pratiği kazanmak; özellikle modern web uygulamalarında bileşen tasarımı, durum yönetimi, sürüm kontrolü ve bulut servislerinin bütünleşik kullanımını deneyimlemektir. Kapsam olarak React tabanlı ön yüz geliştirme, Redux Toolkit/RTK Query ile durum ve veri önbellekleme yönetimi, Firebase üzerinde kimlik doğrulama (Auth), veritabanı (Firestore) ve dosya yönetimi (Storage) süreçleri ile proje yaşam döngüsünün gözlemlenmesi (gereksinim analizi, sprint planlama, günlük toplantılar, kod inceleme ve sürümleme) yer almaktadır. Staj yapılan kurum Sia Teknoloji, iş dünyasına özel yazılımlar geliştiren ve kurumların süreçlerini dijitalleştirmeye odaklanan bir teknoloji şirketidir. Staj süresince Yazılım Geliştirme biriminde görev alınarak; kimlik doğrulamalı bir gönderi (post) uygulamasının arayüz bileşenleri, CRUD akışları, dosya yükleme ilerleme göstergeleri, korumalı rotalar ve liste/detay sayfaları gibi modüllere katkı sağlanacak; ayrıca küçük çaplı iyileştirmeler, hata giderme ve dokümantasyon çalışmaları yürütülecektir. Bu çalışmalar aracılığıyla temiz kod ve yeniden kullanılabilir bileşen geliştirme, tek yönlü veri akışı ve mimari ayrışım, Git/GitHub ile dallanma–PR–code review disiplini, temel erişilebilirlik ve performans iyileştirme yaklaşımları, hata yönetimi ve kullanıcı geri bildirim tasarımı gibi mesleki becerilerin kazanılması hedeflenmekte; takım içinde iletişim, zaman yönetimi ve problem çözme yetkinliklerinin güçlendirilmesi amaçlanmaktadır.

11/08/2025 tarihinden 15/08/2025 tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi	Git kurulumu ve txt-dosyası ile denemeler	8	8
Salı	Merge/Fast-Forward & Conflict Çözümü, Stash/Pop Post App İskeleti Repo'ya Yüklendi	9	8
Çarşamba	Uygulama iskeleti, state ve veriyi üstte toplama	10	8
Perşembe	Controlled input ile veri girişi ve listeleme	11	8
Cuma	Redux'u Anlamaya Başlamak – E ticaret uygulaması	12	8
Cumartesi	*********		
Denetleyenin İmzası		Toplam Saat	40

18/08/2025 tarihinden 22/08/2025 tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi	React router ile sayfaları birbirine bağlamak E ticaret uygulaması	13	8
Salı	Firebase Projesi Açma ve Bağlantı Kurma	14	8
Çarşamba	Sayfaların Oluşturulması ve Router Kurulumu	15	8
Perşembe	Auth Sayfası ve Giriş/Kayıt Akışı	16-17	8
Cuma	Firebase Database ve Storage Entegrasyonu Navbar	18	8
Cumartesi	*********		
Denetleyenin İmzası		Toplam Saat	40

25/08/2025 tarihinden 29/08/2025 tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi	Spinner, Redux Toolkit Kurulumu ve Store Yapısı	19	8
Salı	AddEditBlog Sayfasının Oluşturulması	20	8
Çarşamba	Blog API'den Veri Çekme ve Home Sayfasına Hazırlık	21	8
Perşembe	Home Sayfası ve Blogların Listelenmesi	22	8
Cuma	Blog Güncelleme/Silme ve Detay Sayfası	23-24	8
Cumartesi	*******		
Denetleyenin İmzası		Toplam Saat	40

01/09/2025 tarihinden 05/09/2025 tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi	MyPost Sayfası (Kullanıcıya Özel Gönderiler)	25	8
Salı	RoomDetail Sayfası (Oda Detayları ve Katkılar)	26	8
Çarşamba	Rooms Sayfası (Tüm Odaların Listelenmesi)	27	8
Perşembe	CreateRoom Sayfası (Yeni Oda Açma)	28	8
Cuma	Genel Entegrasyon, ProtectedRoute ve Testler	29	8
Cumartesi			
Denetleyenin İmzası		Toplam Saat	40

...../…../20.… tarihinden …../…../20.… tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi			
Salı			
Çarşamba			
Perşembe			
Cuma			
Cumartesi			
Denetleyenin İmzası		Toplam Saat	

...../…../20.… tarihinden …../…../20.… tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi			
Salı			
Çarşamba			
Perşembe			
Cuma			
Cumartesi			
Denetleyenin İmzası		Toplam Saat	

...../…../20.… tarihinden …../…../20.… tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi			
Salı			
Çarşamba			
Perşembe			
Cuma			
Cumartesi			
Denetleyenin İmzası		Toplam Saat	

...../…../20.… tarihinden …../…../20.… tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi			
Salı			
Çarşamba			
Perşembe			
Cuma			
Cumartesi			
Denetleyenin İmzası		Toplam Saat	

...../…../20.… tarihinden …../…../20.… tarihine kadar bir haftalık çalışma

Gün	Yapılan İşler	Yapılan İşle İlgili Bilginin Bulunduğu Sayfa	Saat
Pazartesi			
Salı			
Çarşamba			
Perşembe			
Cuma			
Cumartesi			
Denetleyenin İmzası		Toplam Saat	

GÜN 1

Git kurulumu ve txt-dosyası ile denemeler

tajın ilk gününde ekip ve süreçlerle tanışma gerçekleştirildi; çalışma koşulları, iletişim kanalları ve teslim standartları hakkında bilgilendirildim. Gün içinde tarafıma React tabanlı **kimlik doğrulama (Auth) giriş ekranı** geliştirme ödevi verildi. Ekranın email/şifre alanları, temel doğrulamalar, kullanıcıya geri bildirim ve başarılı giriş sonrası yönlendirme adımlarını içermesi gerektiği belirtildi. Gerekli ortam kurulumlarını tamamlamak üzere bilgisayarıma Node LTS ve tercih edilen editör eklentileri kuruldu; proje iskeleti oluşturularak 'Auth' sayfası için taslak bir bileşen yapısı hazırlandı. Sürüm kontrolüne hazırlık kapsamında **Git** yazılımı kuruldu, kullanıcı adı ve e-posta tanımları yapıldı ve örnek bir klasörde deneme amaçlı yerel depo başlatıldı. 'notlar.txt' isimli sade bir metin dosyası üzerinde değişiklikler yaparak `status → add → commit` akışını tecrübe ettim; `log` ve `diff` çıktılarıyla değişikliklerin geçmişte nasıl izlendiğini gözlemledim. Günün sonunda hem ödevin kapsamı hem de kullanacağım araçlar netleşmiş, temel geliştirme ve sürüm kontrolü hazırlıkları tamamlanmıştır

```
(base) atilsamancioglu@Atil-MacBook-Pro ~ % pwd
/Users/atilsamancioglu
(base) atilsamancioglu@Atil-MacBook-Pro ~ % ls
AndroidStudioProjects   Documents              Music                  PycharmProjects        VirtualBox VMs
Applications            Downloads              Pictures               Python Projects        ibmapi.txt
Creative Cloud Files    Library                Postman                UnityProjects          instagramproject
Desktop                 Movies                 Public                 Users                  pitchshiftLogFile.txt
(base) atilsamancioglu@Atil-MacBook-Pro ~ % cd Desktop
(base) atilsamancioglu@Atil-MacBook-Pro Desktop % ls
(base) atilsamancioglu@Atil-MacBook-Pro Desktop % mkdir GitKursu
(base) atilsamancioglu@Atil-MacBook-Pro Desktop % ls
GitKursu
(base) atilsamancioglu@Atil-MacBook-Pro Desktop % cd GitKursu
(base) atilsamancioglu@Atil-MacBook-Pro GitKursu % ls
(base) atilsamancioglu@Atil-MacBook-Pro GitKursu % git status
fatal: not a git repository (or any of the parent directories): .git
(base) atilsamancioglu@Atil-MacBook-Pro GitKursu % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /Users/atilsamancioglu/Desktop/GitKursu/.git/
(base) atilsamancioglu@Atil-MacBook-Pro GitKursu %
```

2. Git Komutları Çalışması

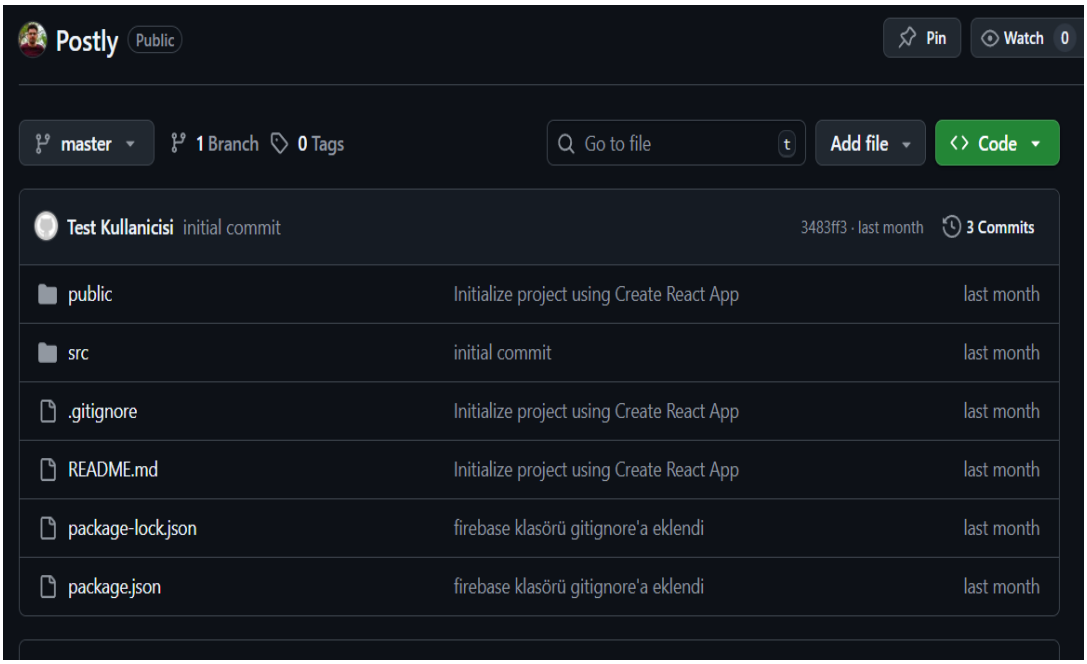
Tarih ve İşyeri Amirinin İmzası	11/08/2025

GÜN 2

Merge/Fast-Forward Conflict Çözümü, Stash/Pop Post App İskeleti Repo'ya Yüklendi

Bugün sürüm kontrol disiplinini derinleştirmek için **HEAD işaretçisi**, **dal (branch) oluşturma**, **merge** çeşitleri ve **geçmişe dönük işlemler** üzerinde uygulamalı çalışma yaptım. Deneme alanı olarak oluşturduğum küçük bir depo içinde `notlar.txt` adlı bir **metin dosyasıyla** ilerledim. Önce `git switch -c feature/deneme` ile yeni bir dal açıp düzenlemeler yaptım; art arda **küçük ve anlamlı commit'ler** oluşturarak `main` dalına **fast-forward merge** senaryosunu doğruladım (doğrudan uçtan uca birleştirme). Ardından, aynı dosyaya `main` ve başka bir feature dalında **eşzamanlı ama çelişen** değişiklikler `git log --graph --decorate --oneline` ile geçmişin **lineer** (fast-forward) ve **dallanmış** (merge commit'li) görünümlerini karşılaştırarak gözlemledim. Yanlış değişiklikleri geri almak için `git restore` / `git checkout -- <dosya>` komutlarını denedim; yarım kalan işleri park edip bağlam değiştirebilmek için `git stash push -m "çalışma taslağı"` kullandım ve işi bitirince `git stash pop` ile geri yükledim.

Çalışmanın sonunda, dallanma stratejisi ve birleştirme türlerinin pratik farklarını netleştirmiş oldum: **fast-forward**'ın temiz geçmiş sağladığını, **merge commit**'in ise karar noktalarını görünür kıldığını deneyimledim. Ayrıca yerel akıştan uzak depoya geçişi doğrulamak üzere **repo'yu düzenleyip ilk yüklemeleri** yaptım; `.gitignore` içeriğini güncelledim, `README.md`'yi ekleyerek proje iskeletini anlaşılır hale getirdim. Böylece ilerleyen günlerde geliştirilecek **Post (gönderi) uygulaması taslağının** dallanma-PR akışına hazır, izlenebilir bir geçmiş üzerinde ilerlemesi için sağlam bir temel oluşturdum.



3. Proje İçin Githup Reposu

Tarih ve İşyeri Amirinin İmzası	12/08/2025

GÜN 3

Uygulama iskeleti, state ve veriyi üstte toplama

Bugün todo uygulamasının temel iskeletini kurdum. Ana bileşen olarak **App.jsx**'i hazırlayıp görevleri tutacağım todos isimli diziyi useState([]) ile tanımladım; yani tüm veriyi tek yerde tuttum

App

. Bu sayede alt bileşenler sadece ekranı gösteriyor, veriyi değiştirme işi her zaman App içinde yapılıyor (kısaca: veriyi en üstte topladım).

Ardından üç küçük parça geliştirdim: **TodoCreate** (yeni görev eklemek için form), **TodoList** (listenin tamamını göstermek için) ve **Todo** (tek bir görevin satırı)

TodoCreate

TodoList

. App içinde ekleme–silme–güncelleme için createTodo, removeTodo ve updateTodo fonksiyonlarını yazdım ve bu fonksiyonları alt bileşenlere **props** olarak gönderdim

App:
Böylece alt tarafta bir butona tıkladığımda, aslında App'teki dizi güncelleniyor ve ekran otomatik yenileniyor.

Liste kısmında TodoList, kendisine gelen todos dizisini .map(...) ile dönüp her bir öğeyi **Todo** bileşeni olarak ekrana bastı; böylece hem kod düzenli kaldı hem de her satır kendi küçük bileşeni içinde yönetildi

TodoList :
Gün sonunda, uygulamanın omurgası tamamlandı: veriyi tek merkezde tutan basit bir yapı kuruldu, alt bileşenler sadece kullanıcı etkileşimini App'e bildiriyor ve ekran temiz bir şekilde güncelleniyor.

```jsx
import './App.css'
import TodoCreate from './components/TodoCreate.jsx'
import TodoList from './components/TodoList.jsx'
import Stack from '@mui/material/Stack';
import { useState } from 'react';
function App() {

  const [todos, setTodos] = useState([]);

  const createTodo = (newTodo) => {
    setTodos([...todos, newTodo]);
  }

  const removeTodo = (todoId) => {
    setTodos([...todos.filter(todo => todo.id !== todoId)])
  }
  const updateTodo = (newTodo) => {
    const updatedTodos = todos.map((todo) => {
      if (todo.id !== newTodo.id) {
        return todo;
      }
      return newTodo;
    })
    setTodos([...updatedTodos]);
  }
  return (
    <div className="App">
      <div stale={{ width: '100%', diplay: 'flex', flexDirection: 'column', alignItems: 'center' }}>
        <Stack direction="row" spacing={2} alignItems="center">
          <TodoCreate onCreateTodo={createTodo} />
        </Stack>
        <TodoList todos={todos} onRemoveTodo={removeTodo} onUpdateTodo={updateTodo} />
      </div>
    </div >


  )
}

export default App
```

4. React Projesi Kaynak Kodları

Tarih ve İşyeri Amirinin İmzası	13/08/2025

GÜN 4

Controlled input ile veri girişi ve listeleme

Bugün uygulamaya görev ekleme özelliğini yaptım. Bunun için ekranda bir **yazı kutusu (input)** ve yanında **Ekle butonu** oluşturdum. Yazı kutusuna yazılan değer, React'in useState özelliği sayesinde anlık olarak bellekte tutuluyor. Yani kullanıcı kutuya her harf yazdığında, bu bilgi doğrudan programa aktarılıyor. Bu yönteme "controlled input" deniyor.

Ekle butonuna basınca yazı kutusundaki metni alıp küçük bir **görev nesnesi** (örneğin {id: 1, content: "Alışveriş yap"}) haline getirdim ve bunu görevler listesine ekledim. Böylece ekrana yeni bir satır olarak yansıdı. Ekleme işlemi bitince kutuyu otomatik temizledim ki yeni görev yazılabilsin.

Listeleme kısmında, eklenen her görevi bir döngüyle (map) tek tek ekrana bastım. Her görev için ayrıca silme ve düzenleme butonları ekledim. Sil butonuna basınca o görev listeden kaldırılıyor, düzenle butonuna basınca da yazı kutusu açılıp görev değiştirilebiliyor.

Son olarak boş liste ve dolu listeyi test ettim. Görev yokken boş ekran, görev ekleyince ise liste halinde görünmesi sağlandı. Böylece kullanıcı için temel görev ekleme, silme ve düzenleme akışı tamamlanmış oldu.

```jsx
import DeleteIcon from '@mui/icons-material/Delete';
import IconButton from '@mui/material/IconButton';
import { FiEdit3 } from "react-icons/fi";
import { FaCheck } from "react-icons/fa6";
import React, { useState } from 'react';
import TextField from '@mui/material/TextField'


function Todo({ todo, onRemoveTodo, onUpdateTodo }) {

    const [editable, setEditable] = useState(false);
    const [newTodo, setNewTodo] = useState(todo.content);

    const removeTodo = () => {
        onRemoveTodo(todo.id);
    }

    const updateTodo = () => {
        if (!newTodo) return;
        const request = {
            id: todo.id,
            content: newTodo,
        };
        onUpdateTodo(request)
        setEditable(false) // Remove the old todo
    }
    return (
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '10px', borderBottom: '1px solid #ccc', marginTop: '15px' }}>
            <div>
                {
                    editable ? <TextField value={newTodo} onChange={(e) => setNewTodo(e.target.value)} fullWidth
                        style={{ width: '300px' }} variant="outlined"
                    /> : todo.content
                }

            </div>
            <div>
                <IconButton aria-label="delete" color="error" size="medium" onClick={removeTodo}>
                    <DeleteIcon />
                </IconButton>
                <IconButton aria-label="delete" color="dark" size="medium">
                    {
                        editable ? <FaCheck onClick={updateTodo} />
                            : <FiEdit3 onClick={() => setEditable(true)} />
                    }

                </IconButton>
            </div>


        </div>


    )
}

export default Todo
```

5. React Projesi Kaynak Kodları

Tarih ve İşyeri Amirinin İmzası	14/08/2025

GÜN 5

Redux'u Anlamaya Başlamak

Bugün daha önce hazırlanmış bir **e-ticaret uygulamasının Redux kısmına** göz attım. Amacım Redux'un mantığını öğrenmekti. Redux'u ilk defa görünce biraz karışık geldi ama yavaş yavaş kavradım. En temel fikir şu: Normalde veriyi useState ile bir bileşende tutuyoruz, ama uygulama büyüdükçe bu veri farklı yerlerde de lazım oluyor. Redux burada devreye giriyor. Veriyi "store" denilen ortak bir yerde saklıyor ve istediğimiz bileşen bu veriye erişebiliyor

Store İncelediğim uygulamada üç tane slice vardı:

- **appSlice**: Uygulamanın genel durumlarını (mesela loading gibi) tutuyor

  appSlice

  .**productSlice**: API'den ürünleri çekiyor, seçilen ürünü kaydediyor

  productSlice

  .**basketSlice**: Kullanıcının sepetini yönetiyor. Ürün ekleme, silme gibi işlemler burada tanımlanmış ve localStorage'a kaydediliyor ki sayfa yenilense bile sepet kaybolmasın

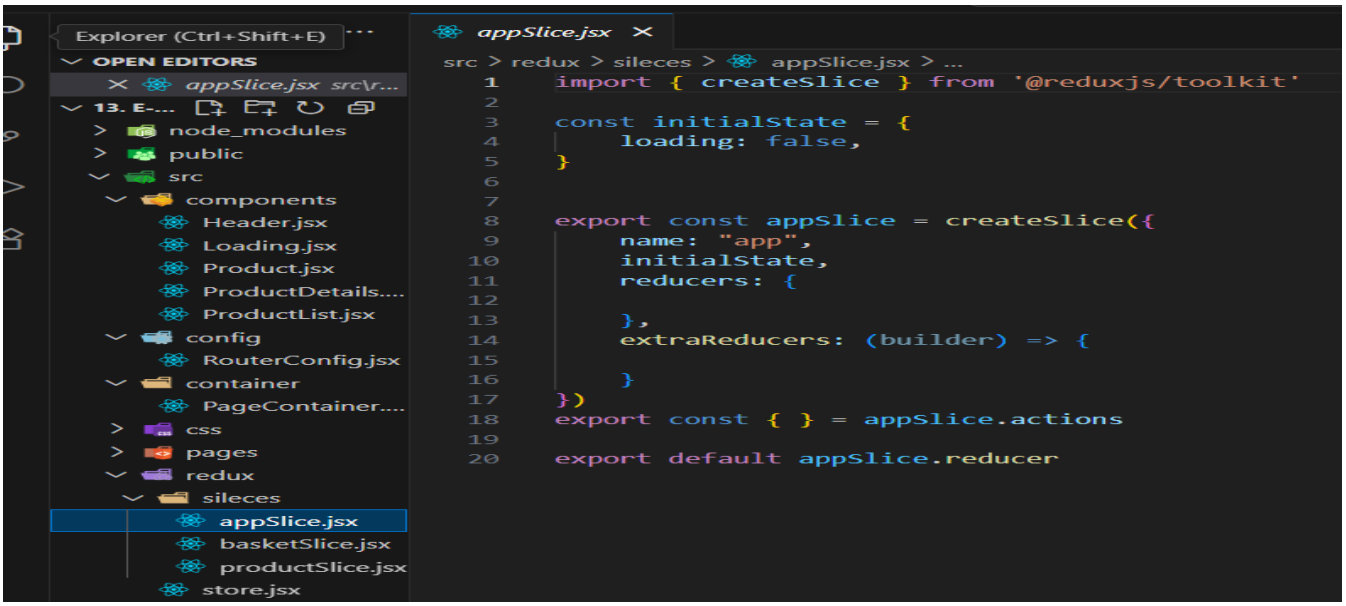
  basketSlice

Bu parçaların hepsi store.jsx dosyasında birleştiriliyor ve configureStore fonksiyonu ile tek bir büyük depoya dönüştürülüyor

store

. Sonrasında uygulamanın istediği yerinde bu store'dan veri çekmek veya güncellemek mümkün hale geliyor.

Bugünün sonunda Redux'un temel mantığını anladım: Uygulamanın beyni gibi çalışan merkezi bir depo var, bütün bileşenler buradan besleniyor. Bu sayede kodlar daha düzenli, veriler daha güvenli tutuluyor.



6. Redux Toolkit Slice Yapısı ve Durum (State) Yönetimi Kodları

Tarih ve İşyeri Amirinin İmzası	15/08/2025

GÜN 6

React router ile sayfaları birbirine bağlamak

Bugün aynı e-ticaret uygulamasında **React Router** kullanımına baktım. Router, uygulamayı farklı sayfalara ayırmak için kullanılıyor. Normalde tek sayfalık uygulama (SPA) yazıyoruz ama kullanıcı farklı adreslere gidiyormuş gibi hissediyor. Bu da react-router-dom sayesinde oluyor.

Örneğin RouterConfig dosyasında şu tanımlar vardı:

- / adresine gidildiğinde **Home** sayfası açılıyor. Burada ürünlerin listesi görünüyor.
- /product-details/:id adresinde ise **ProductDetails** sayfası var. Burada :id dinamik parametre oluyor. Yani hangi ürünün detayına tıklandıysa, onun bilgisi gösteriliyor.

Bu yapı sayesinde kullanıcı ana sayfada ürünleri görüyor, bir ürüne tıkladığında yeni bir sayfaya geçmiş gibi detaylarını inceliyor. Router ile Redux birleşince daha da güçlü oluyor: Çünkü ürünler Redux store'da saklanıyor, router sadece hangi ürünün görüntüleneceğini belirliyor.

Bugünün sonunda şunu anladım: Router sayfalar arasında geçişi sağlıyor, Redux ise sayfaların ihtiyaç duyduğu ortak veriyi tutuyor. İkisini birlikte kullanınca uygulama daha profesyonel bir yapıya kavuşuyor.

```
rc > config > RouterConfig.jsx > default
1  import React from 'react';
2  import { Routes, Route } from 'react-router-dom';
3  import Home from '../pages/Home';
4  import ProductDetails from '../components/ProductDetails';
5
6  function RouterConfig() {
7      return (
8          <Routes>
9              <Route path='/' element={<Home />}></Route>
10             <Route path='/product-details/:id' element={<ProductDetails />}></Route>
11         </Routes>
12     )
13 }
14
15 export default RouterConfig
```

7. React Router ile Sayfa Yönlendirme Yapılandırması

Tarih ve İşyeri Amirinin İmzası	18/08/2025

GÜN 7

Firebase Projesi Açma ve Bağlantı Kurma

Bugün uygulamamda kullanıcı girişi yapabilmek için Firebase üzerinde yeni bir proje oluşturdum. İlk olarak Firebase hesabı açtım ve konsolda "firebase" isminde bir proje başlattım. Proje oluşturulurken sadece **Authentication** kısmını aktif ettim. Çünkü başlangıçta hedefim kullanıcıların kayıt olup giriş yapabilmesi, diğer kısımlar (Database, Storage) daha sonra devreye girecek.

Proje oluşturulduktan sonra Firebase'in bana verdiği özel yapılandırma ayarlarını (apiKey, authDomain, projectId, appId vb.) aldım. React tarafında src klasörüne **firebase.js** dosyası açtım ve terminalden npm install firebase komutunu çalıştırarak paketi projeme ekledim. Ardından firebase/app modülünden initializeApp fonksiyonunu çağırıp config bilgilerini buraya yapıştırdım.

Authentication için ayrıca getAuth fonksiyonunu import ederek uygulamamda dışarıya export ettim. Bu sayede ilerleyen günlerde login/register ekranında bu auth yapısını kullanarak kullanıcı giriş ve kayıt işlemlerini gerçekleştirebileceğim.

Bugünün sonunda elimde Firebase ile bağlantısı kurulmuş, Authentication özelliği aktif edilmiş bir proje oldu. Artık bundan sonra uygulama tarafında giriş ekranı hazırlayıp Firebase Auth ile bağlamaya hazırım.

```
firebase.js > ...
// src/firebase.js
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";
import { getStorage } from "firebase/storage";

const firebaseConfig = {
  apiKey: "AIzaSyDUPsm3fMByhYkQf6oFO76UCf3X8oh3fg8",
  authDomain: "fir-7fdfd.firebaseapp.com",
  projectId: "fir-7fdfd",
  storageBucket: "fir-7fdfd.firebasestorage.app",
  messagingSenderId: "898083675526",
  appId: "1:898083675526:web:222f80cf41b9ddd82ea007"
};


export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);     // <-- Firestore'u da dışa aktar (ileride lazım)
export const storage = getStorage(app);  // <-- HATAYI ÇÖZEN EXPORT
```

8. Firebase Bağlantı ve Yapılandırma Dosyası (firebase.js)

Tarih ve İşyeri Amirinin İmzası	19/08/2025

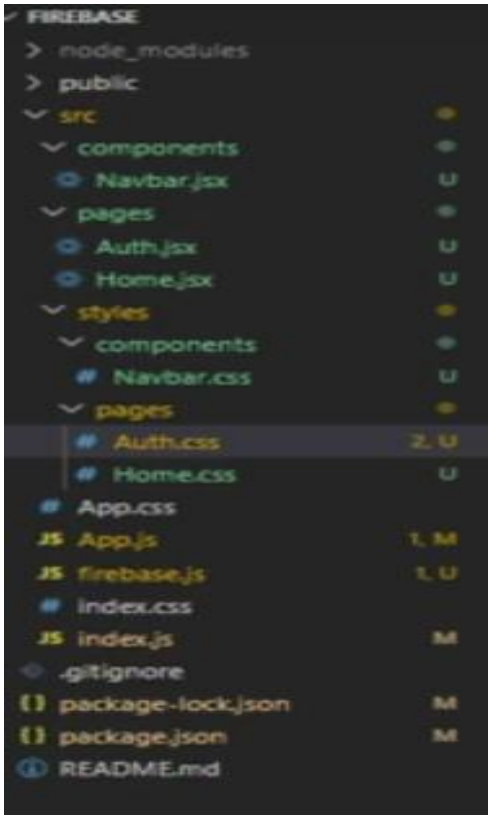
GÜN 8

Sayfaların Oluşturulması ve Router Kurulumu

Bugün projeyi sayfalara ayırmaya başladım. İlk iş olarak terminalden npm install react-router-dom kurdum. Daha sonra src altında **pages** klasörü açıp içine **Home.jsx** ve **Auth.jsx** dosyalarını ekledim. Uygulamanın giriş noktasına BrowserRouter sarmalaması yaptım ve ayrıca ayrı bir RouterConfig bileşeni açıp rotaları buradan yönettim.

Routes / Route yapısını kullanarak iki temel rota tanımladım: / adresinde **Home**, /auth adresinde ise **Auth** sayfası çalışıyor. İlk testte /auth yazınca boş Auth sayfam açıldı, / yazınca Home geldi. Navbar ekleyerek Home ve Login/Register arasında linklerle geçiş yapmayı sağladım.

Home sayfasına şimdilik "Anasayfa – postları burada listeleyeceğim" diye basit bir placeholder yazdım. Auth sayfasını ise boş bıraktım, sadece başlık ve form için alan açtım. Bugün temel iskeleti çıkardım, artık proje sayfa geçişleri olan bir yapıya kavuştu.



9. Dosya Hiyerarşisi ve App.js Konfigürasyonu

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Navbar from './components/Navbar';
import Auth from './pages/Auth';
import Home from './pages/Home';
function App() {
  return (
    <div >
      <BrowserRouter>
        <Navbar/>        You, 1 second ago • Uncommitted changes
        <Routes>
          <Route path="/" element={<Auth />} />
          <Route path="dashboard" element={<Home />} />
        </Routes>
      </BrowserRouter>
      </div>
  );
}

export default App;
```

Tarih ve İşyeri Amirinin İmzası	20/08/2025

GÜN 9

Auth Sayfası ve Giriş/Kayıt Akışı

Bugün Auth sayfasına yoğunlaştım. Öncelikle Auth.css dosyası açıp giriş ekranının tasarımını basit CSS ile yaptım. Ortada kutucuk şeklinde bir alan, içinde başlık, email ve şifre girişi olacak şekilde düzenledim.

Sonra kod tarafında useState ile email ve şifreyi tuttum. Formdaki input alanlarını controlled hale getirip onChange ile state'i güncelledim

Auth :Ayrıca sayfada iki mod var: **Register** ve **Login**. Kullanıcı "daha önce kayıt oldunuz mu?" yazısına tıklayınca mod değişiyor.

Butona basıldığında authFunc çalışıyor. Eğer email ya da şifre boşsa toast.error("Email ve şifre girilmeli") hatası veriyor. Şifre 6 karakterden kısa ise yine uyarı çıkıyor. Her şey uygunsa:

Kayıt için createUserWithEmailAndPassword(auth, email, password) çağrılıyor. Başarılı olursa /dashboard sayfasına yönlendiriyor.

Giriş için signInWithEmailAndPassword çalışıyor, o da başarılı olursa yine dashboard'a yönlendiriyor.

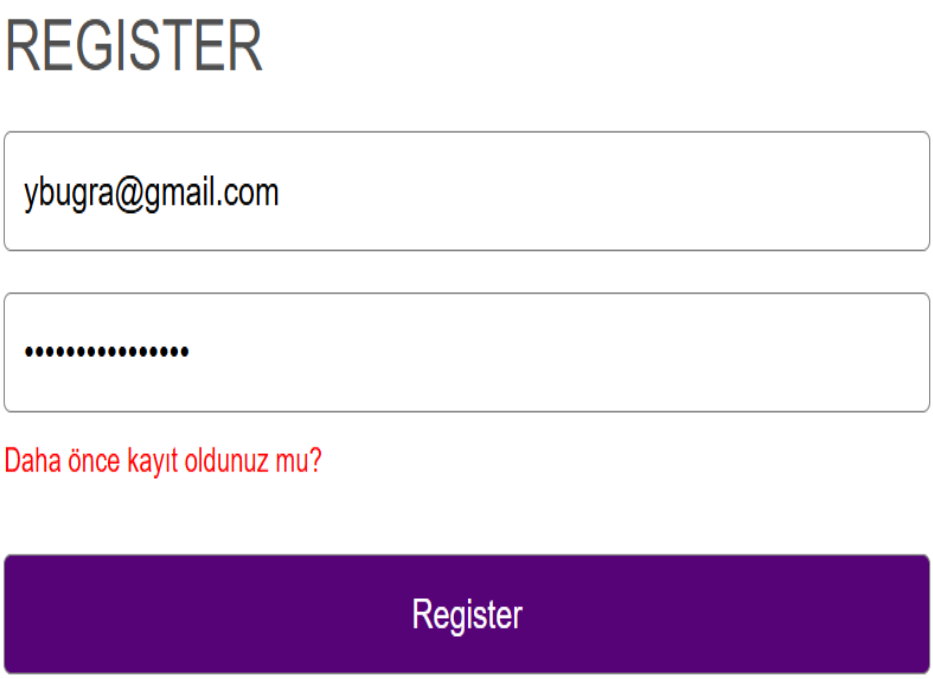
Yanlış bilgi girilirse Firebase'in verdiği hata mesajı ekrana toast ile yansıyor. Böylece kullanıcı yanlış yaptığında hemen geri bildirim alıyor.

Bugünü bitirdiğimde elimde çalışan bir giriş/kayıt ekranı oldu. UI basit ama işlevsel. Email ve şifre ile hesap açılabiliyor ve giriş yapılabiliyor.

```
import React, { useState } from 'react';
import '../styles/pages/Auth.css';
import { createUserWithEmailAndPassword, signInWithEmailAndPassword } from 'firebase/auth';
import { auth } from '../firebase';
import { toast } from 'react-toastify';

const Auth = () => {
  const [signUp, setSignUp] = useState(true);
  const [authData, setAuthData] = useState({ email: '', password: '' });

  const onChangeFunc = (e) => {
    setAuthData({ ...authData, [e.target.name]: e.target.value });
  };
      const authFunc: () => Promise<void>
 const authFunc = async () => {
  if (!authData.email || !authData.password) {
    toast.error("Email ve şifre girilmeli");
    return;
  }
  if (authData.password.length < 6) {
    toast.error("Şifre en az 6 karakter olmalı");
    return;
    }

  try {
    if (signUp) {
      const data = await createUserWithEmailAndPassword(auth, authData.email, authData.password);
      console.log("signup success", data);
      if (data.user) window.location = "/dashboard";
    } else {
      const data = await signInWithEmailAndPassword(auth, authData.email, authData.password);
      console.log("login success", data);
      if (data.user) window.location = "/dashboard";
    }
  } catch (error) {
    // Tüm hata objesini console'a bas
    console.error("Firebase Auth Hata:", error);
    // SDK hata kodunu göster
    console.log("error.code:", error.code);
    toast.error(error.message || "Bir hata oluştu");
  }
};

  return (
    <div className="auth">
      <div className="auth-container">
        <h2>{signUp ? 'REGISTER' : 'LOGIN'}</h2>
        <input name="email" value={authData.email} onChange={onChangeFunc} type="email" placeholder="Email" />
        <input name="password" value={authData.password} onChange={onChangeFunc} type="password" placeholder="Password" />

        <p onClick={(() => setSignUp(!signUp))}>
          {signUp ? 'Daha önce kayıt oldunuz mu?' : 'Kayıt olmak mı istiyorsun?'}
        </p>
        <div onClick={authFunc} className="auth-container-button">
          {signUp ? 'Register' : 'Login'}
        </div>
      </div>
    </div>
  );
};

export default Auth;
```

10. Auth Bileşeni ve Firebase Giriş/Kayıt Kodları

Tarih ve İşyeri Amirinin İmzası	



11. Kullanıcı Kayıt (Register) Ekranı Arayüzü

Tarih ve İşyeri Amirinin İmzası	21/08/2025

GÜN 10

Firebase Database ve Storage Entegrasyonu Navbar

Bugün giriş ekranını bitirdikten sonra bana yeni bir görev verildi: artık kullanıcıların sadece giriş yapması değil, aynı zamanda **post paylaşabilmesi** gerekiyordu. Bunun için Firebase'de daha önce aktif ettiğim **Authentication**'ın yanına bu sefer **Firestore Database** ve **Firebase Storage**'ı ekledim.

Firestore'u gönderilerin başlık ve açıklama bilgilerini tutmak için hazırladım.

Storage tarafını da fotoğraf yükleme işlemleri için aktif ettim. Böylece her gönderinin hem yazı hem de görseli olacak şekilde kaydedilebilmesini sağladım.

Ardından sayfaları daha düzenli ve modern gösterebilmek için **MDB React UI Kit** paketini yükledim. Bu UI kit sayesinde kart yapıları, butonlar ve navbar tasarımları çok daha kolay oldu. UI kit'in dökümantasyonunu biraz araştırarak temel bileşenleri nasıl kullanabileceğimi öğrendim.

Daha sonra uygulamaya yeni sayfalar ekledim: **Create, Update, Detail ve Home**. Bu sayfalar arasında geçiş yapabilmek için App.js içinde **react-router-dom** kullanarak linkler oluşturdum. Örneğin, Home'dan Create sayfasına geçmek veya bir postun detayına gitmek için Router yapılarını kurdum.

Son olarak, uygulamanın üst kısmına bir **Navbar** ekledim. Navbar içinde "Home" ve "Create" linklerini koydum, böylece kullanıcı tek tıkla sayfalar arasında geçiş yapabiliyor. Bu kısımda UI kit'ten hazır Navbar bileşenini kullanarak daha şık bir görünüm elde ettim.

Bugünün sonunda elimde, Firebase Database ve Storage'a bağlanan ve farklı sayfaları olan temel bir blog/post uygulaması iskeleti oluşmuş oldu.

```
10
11  const Navbar = () => {
12    const navigate = useNavigate();
13    return (
14      <MDBNavbar expand="lg" dark bgColor="dark">
15        <MDBContainer fluid>
16          <MDBNavbarBrand tag="span" className="mb-0 h1 fw-bold">
17            RTK-Query Firebase Blog App
18          </MDBNavbarBrand>
19          <div style={{ float: "right", marginRight: "50px" }}>
20            <MDBNavbarNav className="mb-2 mb-lg-0" fullWidth={false}>
21              <MDBNavbarItem>
22                <p className="header-text" onClick={() => navigate("/")}>
23                  Home
24                </p>
25              </MDBNavbarItem>
26              <MDBNavbarItem>
27                <p className="header-text" onClick={() => navigate("/create")}>
28                  Create
29                </p>
30              </MDBNavbarItem>
31            </MDBNavbarNav>
32          </div>
33        </MDBContainer>
34      </MDBNavbar>
35    );
36  };
37
38  export default Navbar;
```

12..MDB React Navbar Kodları ve Menü Görünümü

Tarih ve İşyeri Amirinin İmzası	22/08/2025

GÜN 11

Spinner, Redux Toolkit Kurulumu ve Store Yapısı

Bugün uygulamaya yeni özellikler eklemeye devam ettim. Öncelikle, veri yüklenirken kullanıcıya boş bir ekran göstermemek için **Spinner** adlı bir component oluşturdum. Bu component, basitçe dönen bir yükleniyor işareti çıkarıyor. Böylece kullanıcı, örneğin blog verileri Firebase'den gelirken uygulamanın donduğunu değil, arka planda işlem yaptığını görebiliyor.

Daha sonra Firebase'den veri çekme, ekleme ve silme işlemlerini daha düzenli yönetebilmek için **Redux Toolkit** kütüphanesini projeye dahil ettim (npm install @reduxjs/toolkit react-redux). Redux Toolkit sayesinde uygulamanın state yönetimini daha kontrollü yapabiliyorum. Bunun için services klasörü altında **BlogsApi.jsx** dosyasını oluşturdum. Burada Firebase'e bağlanan sorguları tanımlayıp, blogları **listeleme, ekleme, güncelleme ve silme** işlemlerini merkezi bir şekilde kontrol edeceğim.

Redux yapısını kurmak için ayrıca bir **store.js** dosyası açtım. Store, uygulamadaki bütün verilerin toplandığı ana merkez gibi çalışıyor. Burada oluşturduğum blogsApi.reducer gibi kısımları ekledim ve middleware ayarlarını yaptım.

Son adım olarak, uygulamanın en üst seviyesindeki index.js dosyasında, **Provider** ile App.js'i sardım. Provider, store'u tüm uygulamaya tanıtmamı sağladı. Böylece artık herhangi bir component içinde direkt Redux Toolkit sorgularını kullanabilirim.

Bugünün sonunda elimde şunlar oldu:

- Yükleme durumlarını gösterecek **Spinner componenti**,
- Firebase sorgularını çalıştırmak için **BlogsApi.jsx**,
- Bütün uygulamayı yönetecek **store.js**,
- Ve store'u tüm uygulamaya tanıtan **Provider** yapısı.

Bu adımlarla beraber uygulama daha kurumsal bir hale gelmeye başladı, veriler artık tek bir merkezden yönetiliyor ve kullanıcıya daha düzgün bir deneyim sunuluyor.

```
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <Provider>
      <App />
    </Provider>
  </React.StrictMode>
);
```

```
components > Spinner.jsx > default
import React from 'react'
import { MDBSpinner } from 'mdb-react-ui-kit'


function Spinner() {
  return (
    <div>
<MDBSpinner
className="me-2 mt-5"
style={{width:"3rem", height:"3rem"}}
>
<span className="visually-hidden">Loading...</span>
</MDBSpinner>
    </div>
  )
}

export default Spinner
```

13. Redux Provider Entegrasyonu ve Spinner Bileşeni Kodları

Tarih ve İşyeri Amirinin İmzası	25/08/2025

GÜN 12

AddEditBlog Sayfasının Oluşturulması

Bugün uygulamada gönderi ekleme ve güncelleme kısmı için **AddEditBlog.jsx** dosyasında çalıştım. İlk olarak formda kullanacağım alanları hazırladım. Bunun için useState ile başlık (title), açıklama (description) ve dosya (file) bilgilerini tuttum. Ayrıca kayıt sırasında butonun durumunu kontrol edebilmek için progress adında bir state daha ekledim.

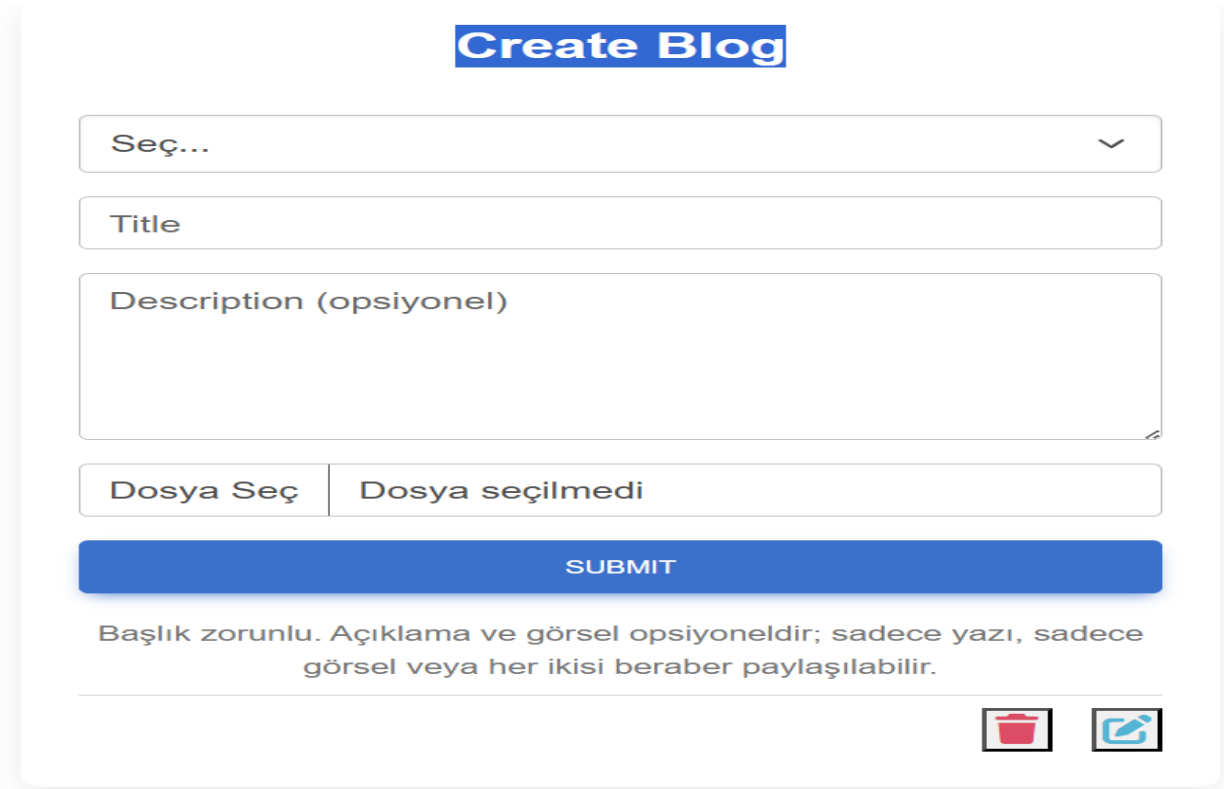
Formun tasarımını kolay yapmak için **MDB React UI Kit**'ten yararlandım. Bu kütüphane ile hazır gelen kart, input, textarea ve butonları kullandım. Formun içinde şunlar yer aldı:

- Kategori seçimi için bir **select** kutusu,
- Başlık girişi için **input**,
- Açıklama için **textarea**,
- Resim yüklemek için **file input**,
- Ve en altta kaydetmek için **submit butonu**.

Resim yükleme kısmını da ekledim. Dosya seçildiğinde Firebase Storage'a gönderiliyor ve yükleme sırasında ekranda **progress bar** çıkıyor. Böylece kullanıcı resmin ne kadar yüklendiğini yüzde olarak görebiliyor. Yükleme tamamlanınca resmin linki (imgURL) alınıyor ve veriye ekleniyor.

Butona basıldığında handleSubmit çalışıyor. Burada önce doğrulama yaptım: başlık boşsa kayıt olmuyor, açıklama ve resimden en az birinin dolu olması gerekiyor. Kullanıcı bir kategori seçmeden de kaydedemiyor. Her şey doğruysa veriler Firestore'a ekleniyor veya mevcut gönderiyse güncelleniyor. Başarılı olursa ekranda toast.success ile "Kaydedildi" ya da "Güncellendi" mesajı görünüyor.

Bugünün sonunda, kullanıcıların başlık, açıklama ve resim ekleyerek gönderi oluşturabileceği, ayrıca isterse var olan gönderisini güncelleyebileceği bir ekran hazır hale geldi. Tasarım da UI kit sayesinde modern ve anlaşılır görünüyor.



14. Blog Gönderisi Ekleme ve Düzenleme Formu Arayüzü

Tarih ve İşyeri Amirinin İmzası	26/08/2025

GÜN 13

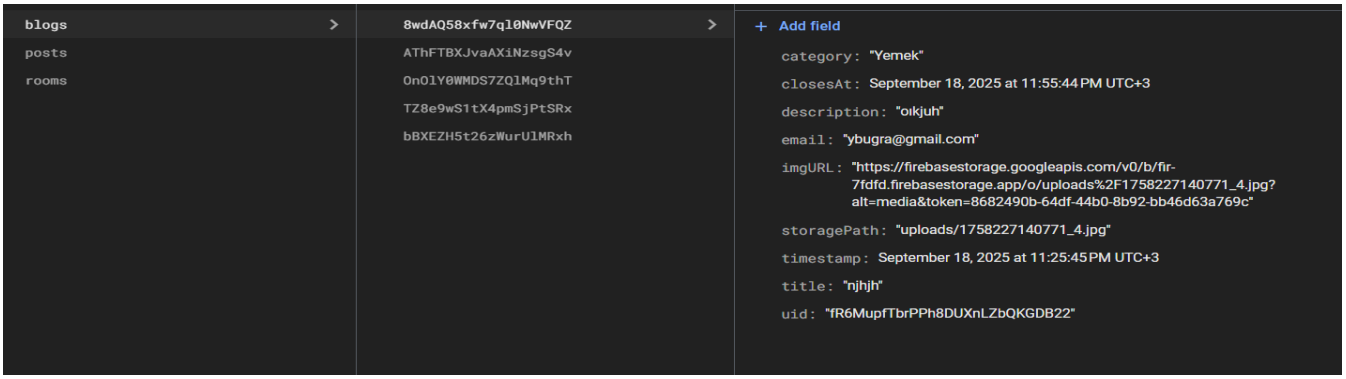
Blog API'den Veri Çekme ve Home Sayfasına Hazırlık

Bugün uygulamayı biraz daha ileriye taşıdım. Artık kullanıcıların yazdığı gönderileri Firebase'den çekmem gerekiyordu. Bunun için services klasöründe **BlogsApi.jsx** dosyasını hazırladım. Burada Firebase'de açtığım blogs koleksiyonuna bağlandım ve yazıları buradan alacak yapıyı kurdum

İlk deneme olarak tüm gönderileri sıralı bir şekilde çekmeye odaklandım. Yani veriler en yeni tarihli olan en üstte görünecek şekilde geliyor. Daha sonra ihtiyaca göre kategorisine göre filtreleme veya kullanıcının kendi yazılarını görmesi gibi seçenekler de eklenebiliyor. Ama bugünkü hedefim temel veri akışını görmekti.

Bu yapıyı kurduktan sonra artık **Home.jsx** sayfasının tasarımına başladım. Yani veriyi sadece konsolda görmek değil, ekranda liste halinde göstermek için hazırlık yaptım. Şimdilik sadece iskeletini kurdum ama artık API'den veri geldiği için listeleme kısmına geçiş yapabileceğim.

Bugünün sonunda elimde Firebase'den veri çekebilen ve Home sayfasına aktarılmaya hazır bir API yapısı oluştu. Yarın bu verileri ekrana kart şeklinde gösterecek ve listeleme tasarımına odaklanacağım.



15. Firestore Veritabanı Görünümü ve Veri Çekme (API) Kodları

```
// Tüm gönderiler (Home - Tümü)
fetchBlogs: builder.query({
  async queryFn() {
    try {
      const qy = query(collection(db, "blogs"), orderBy("timestamp", "desc"));
      const snap = await getDocs(qy);
      return { data: snap.docs.map(mapDoc) };
    } catch (err) {
      return { error: { message: err?.message || String(err) } };
    }
  },
  providesTags: (result) =>
    result
      ? [
          ...result.map(({ id }) => ({ type: "Blogs", id })),
          { type: "Blogs", id: "LIST" },
        ]
      : [{ type: "Blogs", id: "LIST" }],
}),

// E-mail'e göre gönderiler (isteğe bağlı)
fetchBlogsByEmail: builder.query({
  async queryFn(email) {
    try {
      if (!email) return { data: [] };
      const qy = query(
        collection(db, "blogs"),
        where("email", "==", email),
        orderBy("timestamp", "desc")
      );
      const snap = await getDocs(qy);
      return { data: snap.docs.map(mapDoc) };
    } catch (err) {
      return { error: { message: err?.message || String(err) } };
    }
  },
  providesTags: (result) =>
    result
      ? [
          ...result.map(({ id }) => ({ type: "Blogs", id })),
          { type: "Blogs", id: "LIST" },
        ]
      : [{ type: "Blogs", id: "LIST" }],
}),
```

Tarih ve İşyeri Amirinin İmzası	27/08/2025

GÜN 14

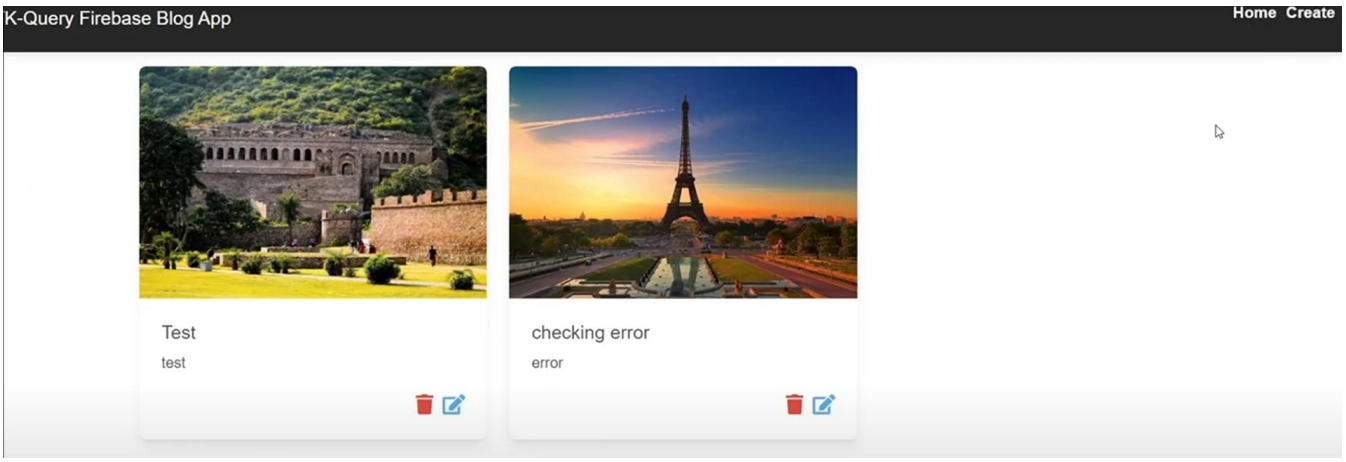
Home Sayfası ve Blogların Listelenmesi

Bugün artık API'den gelen verileri ekranda göstermek için **Home.jsx** üzerinde çalıştım. Daha önce BlogsApi.jsx içinde hazırladığım fonksiyonları bu sayfaya bağladım ve **useFetchBlogsQuery** hook'unu kullanarak Firebase'den gönderileri çektim.

İlk etapta verileri konsolda görüyordum, bugün onları kartlar halinde sayfaya bastım. MDB React UI Kit'in hazır kart yapısını kullandım. Her kartta gönderinin resmi, başlığı ve açıklaması yer aldı. Altta da küçük ikonlar ekledim: çöp kutusu ile **silme**, kalem ikonu ile **düzenleme** işlemleri yapılabiliyor.

Tasarımı ilk aşamada sade tuttum. Kartlar yan yana diziliyor ve boş bir ekran yerine artık gerçek gönderiler görünüyor. Bu haliyle basit ama işlevsel bir görünüm oldu. İleride kategori filtreleme, arama ya da farklı düzenler ekleyerek daha da geliştirmeyi düşünüyorum.

Bugünün sonunda elimde çalışan bir **Home sayfası** oldu. Artık kullanıcıların eklediği post'lar veritabanından çekilip anasayfada kart olarak listeleniyor. Bu kısım uygulamanın en temel parçalarından biri oldu.



16. Home Sayfası ve Blog Listeleme Arayüzü

Tarih ve İşyeri Amirinin İmzası	28/08/2025

GÜN 15

Blog Güncelleme/Silme ve Detay Sayfası

Bugün öncelikle **BlogsApi** dosyasında güncelleme ve silme işlemlerini tamamladım. Güncelleme kısmında id'nin doğru gelip gelmediğini kontrol ettim, ardından Firestore'daki belgeyi updateDoc ile güncelledim. Ayrıca kaydın ne zaman değiştiğini tutmak için updatedAt alanına server timestamp ekledim. Silme işleminde de benzer şekilde id kontrolü yaptım ve deleteDoc ile kaydı tamamen kaldırdım. Burada RTK Query'nin invalidatesTags özelliği önemli oldu; bu sayede güncelleme veya silme işleminden sonra liste kendiliğinden yenilendi, sayfayı manuel olarak yenilemeye gerek kalmadı.

Ardından **Detail.jsx** sayfasını yazdım. Bu sayfa tek bir gönderiyi detaylı şekilde göstermek için hazırlandı. URL'den gelen id'yi alıp ilgili veriyi çektim. Veri yüklenirken spinner görünüyor, hata olursa kullanıcıya uyarı mesajı çıkıyor. Görsel bulunmadığında boş kalmaması için placeholder ekledim. Tarih alanını Firestore'dan çekip toLocaleString ile okunabilir hale getirdim. Tasarımı sade tuttum: üstte görsel, altında başlık, açıklama ve tarih bilgisi olacak şekilde basit bir düzen kurdum.

Sonuç olarak, güncelleme ve silme akışları tamamlandı, tek gönderi için detay sayfası hazır hale geldi. Böylece uygulamanın listeleme, güncelleme, silme ve detay ekranı birbirine bağlanmış oldu.

```
    // Update
    updateBlog: builder.mutation({
      async queryFn({ id, updates }) {
        try {
          if (typeof id !== "string")
            return { error: { message: "updateBlog: id string olmalı" } };
          const ref = doc(db, "blogs", id);
          await updateDoc(ref, { ...updates, updatedAt: serverTimestamp() });
          return { data: "ok" };
        } catch (err) {
          return { error: { message: err?.message || String(err) } };
        }
      },
      invalidatesTags: (r, e, { id }) => [
        { type: "Blogs", id },
        { type: "Blogs", id: "LIST" },
      ],
    }),

    // Delete
    deleteBlog: builder.mutation({
      async queryFn(id) {
        try {
          if (typeof id !== "string")
            return { error: { message: "deleteBlog: id string olmalı" } };
          await deleteDoc(doc(db, "blogs", id));
          return { data: "ok" };
        } catch (err) {
          return { error: { message: err?.message || String(err) } };
        }
      },
      invalidatesTags: (r, e, id) => [
        { type: "Blogs", id },
        { type: "Blogs", id: "LIST" },
      ],
    }),
  }),
});
```

Tarih ve İşyeri Amirinin İmzası	

```jsx
// src/pages/Detail.jsx
import React, { useEffect } from "react";
import { useParams } from "react-router-dom";
import { toast } from "react-toastify";
import { skipToken } from "@reduxjs/toolkit/query";
import { useFetchBlogQuery } from "../services/BlogsApi";
import {
  MDBCard, MDBCardBody, MDBCardTitle, MDBCardText
} from "mdb-react-ui-kit";
import Spinner from "../components/Spinner";

const fallback = "https://placehold.co/1200x600?text=No+Image";

const Detail = () => {
  const { id } = useParams();
  const { data: blog, isLoading, isError, error } = useFetchBlogQuery(id ?? skipToken);

  useEffect(() => {
    if (isError) toast.error(error?.message || "Blog yüklenemedi");
  }, [isError, error]);

  if (!id) return <div>Geçersiz istek: id yok.</div>;
  if (isLoading) return <Spinner />;

  return (
    <MDBCard className="mb-3">

      {/* --- GÖRÜNTÜ KUTUSU --- */}
      <div
        style={{
          width: "100%",
          maxHeight: "80vh",        // ekrana göre sınır
          background: "#f3f4f6",    // letterbox alanı
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          overflow: "hidden",
        }}
      >
        <img
          src={blog?.imgURL || fallback}
          alt={blog?.title || "Image"}
          onError={(e) => (e.currentTarget.src = fallback)}
          loading="lazy"
          style={{
            // resmin TAMAMI görünsün (kırpma yok)
            objectFit: "contain",
            objectPosition: "center",
            // boyutu kutuya göre ayarla
            maxWidth: "100%",
            maxHeight: "80vh",
            width: "auto",
            height: "auto",
            display: "block",
          }}
        />
      </div>

      <MDBCardBody>
        <MDBCardTitle className="h4 fw-bold">{blog?.title}</MDBCardTitle>

        {blog?.timestamp && (
          <MDBCardText className="text-muted">
            {typeof blog.timestamp?.toDate === "function"
              ? blog.timestamp.toDate().toLocaleString()
              : new Date(blog.timestamp).toLocaleString()}
          </MDBCardText>
        )}

        <MDBCardText className="text-start">{blog?.description}</MDBCardText>
      </MDBCardBody>
    </MDBCard>
  );
};

export default Detail;
```

17. Blog Güncelleme/Silme Fonksiyonları ve Detay Sayfası Kodları

Tarih ve İşyeri Amirinin İmzası	29/08/2025

GÜN 16

MyPost Sayfası (Kullanıcıya Özel Gönderiler)

Bugün “MyPost.jsx” sayfasını ele aldım. Bu ekranın amacı, giriş yapmış kullanıcının sadece kendi oluşturduğu gönderileri görebilmesi. Bunun için kullanıcıya ait UID üzerinden Firestore’daki verileri filtreledim. Listeleme kısmında kart yapısını kullandım. Her kartta gönderinin başlığı, resmi, kısa açıklaması ve eklenme tarihi görünüyor.

Ekstra olarak, bu ekranda **silme ve düzenleme işlemlerini** sadece gönderinin sahibi yapabiliyor. Bunun için kartlara iki buton ekledim: “Sil” ve “Düzenle”. Silme butonuna basıldığında ilgili belgenin Firestore’daki kaydı siliniyor, aynı zamanda Firebase Storage’daki resmi de kaldırılıyor. Kullanıcıya işlem sonucunu göstermek için toast.success veya toast.error mesajları ekledim.

Düzenleme kısmında ise “Düzenle” butonuna basıldığında **AddEditBlog.jsx** sayfasına yönlendirme yaptım. Bu sayfa, gönderi ID’siyle birlikte açılıyor ve form otomatik olarak o gönderinin bilgileriyle doluyor. Böylece kullanıcı ister sadece metni değiştiriyor isterse resmi güncelliyor.

Bu sayfanın testini birkaç deneme hesabı ile yaptım. Kendi gönderilerimde silme/düzenleme butonlarının göründüğünü, ama başkasının gönderilerinde görünmediğini teyit ettim. Bu sayede uygulama kişisel bir alan hissi verdi ve kullanıcı kendi içeriklerini yönetebilir hale geldi.

```
pages > MyPost.jsx > MyPost
// src/pages/MyPost.jsx
import React, { useEffect } from "react";
import Spinner from "../components/Spinner";
import {
  useFetchBlogsByUidQuery,
  useDeleteBlogMutation,
} from "../services/BlogsApi";
import { Link } from "react-router-dom";
import { toast } from "react-toastify";
import {
  MDBRow, MDBCol, MDBCard, MDBCardImage, MDBCardBody,
  MDBCardTitle, MDBCardText, MDBBtn, MDBIcon,
} from "mdb-react-ui-kit";

const placeholder = "https://via.placeholder.com/800x450?text=No+Image";

const excerpt = (str = "", count = 80) =>
  str.length > count ? `${str.slice(0, count)}...` : str;

const MyPost = ({ users }) => {
  const email = users?.email || null; // alt bilgi için
  const uid   = users?.uid   || null; // 🔑 sorgu için

  const { data = [], isLoading, isError, error } =
    useFetchBlogsByUidQuery(uid, { skip: !uid });

  const [deleteBlog] = useDeleteBlogMutation();

  useEffect(() => {
    if (isError) toast.error(error?.message || "Bir hata oluştu");
  }, [isError, error]);

  if (!uid) return <div style={{ padding: 16 }}>Giriş yapılmadı.</div>;
  if (isLoading) return <Spinner />;

  const handleDelete = async (id) => {
    if (!window.confirm("Emin misin ?")) return;
    try {
      await deleteBlog(id).unwrap();
      toast.success("Blog deleted successfully");
    } catch (e) {
      toast.error(e?.message || "Silme başarısız");
    }
  };

```

18. Kullanıcıya Özel Gönderileri Listeleyen MyPost Bileşeni Kodları

Tarih ve İşyeri Amirinin İmzası	01/09/2025

GÜN 17

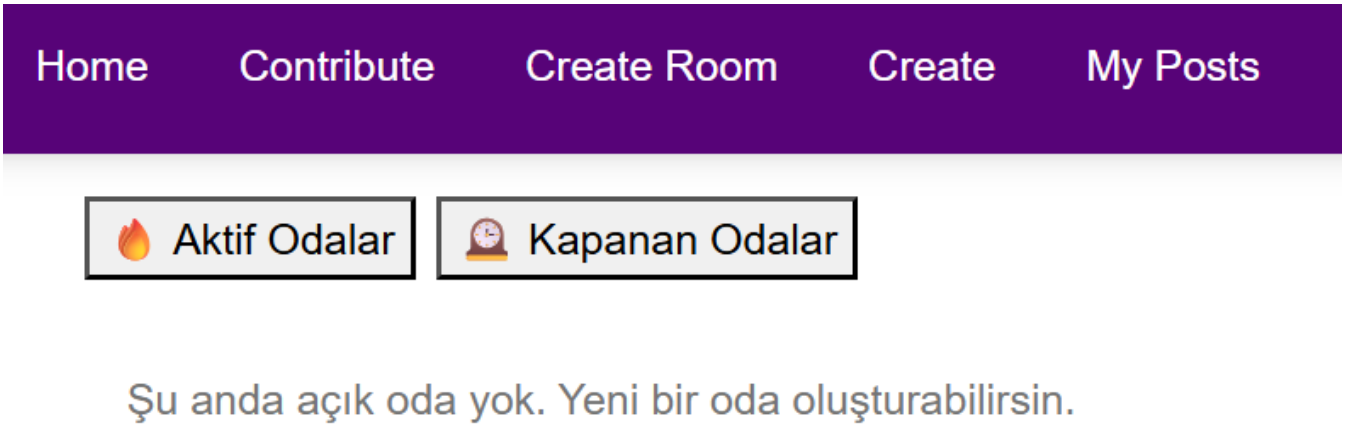
RoomDetail Sayfası (Oda Detayları ve Katkılar)

Bugün “RoomDetail.jsx” üzerinde çalıştım. Oda detay ekranında seçilen odanın tüm bilgilerini gösterecek şekilde bir yapı kurdum. Kartın üst kısmında odanın başlığı ve açıklaması var. Eğer oda açık durumdaysa ekranda geri sayım (Countdown bileşeni) çalışıyor. Oda kapanmışsa artık sadece “Arşivlendi” yazısı görünüyor.

Oda detayında en önemli kısım, kullanıcıların katkı (contribute) yapabilmesi oldu. Eğer kullanıcı giriş yaptıysa altta katkı formu görünüyor. Buradan ister kısa bir yazı eklenebiliyor isterse resim yüklenebiliyor. Resim seçildiğinde yükleme işlemi Firebase Storage’a yapılıyor ve yükleme sırasında progress bar ile yüzde gösteriliyor. Yükleme bitince getDownloadURL() ile link alınıyor ve Firestore’daki oda dökümanına ekleniyor.

Katkı başarıyla kaydedildiğinde “Katkı paylaşıldı” mesajı çıkıyor. Hata olursa da kullanıcıya açıklayıcı bir hata mesajı gösterdim. Böylece odalar sadece tek bir kişinin açtığı alan olmaktan çıktı, toplu etkileşime açık bir platform haline geldi.

Son olarak, yapılan katkılar liste halinde odanın altında görünüyor. Katkı yapan kişinin ismi ve katkı içeriği ekranda sıralanıyor. Bu sayede oda mantığı gerçek anlamda “katkı toplama” işlevine kavuştu.



19. Oda Detayları ve Katkı (Contribute) Arayüzü

Tarih ve İşyeri Amirinin İmzası	02/09/2025

GÜN 18

Rooms Sayfası (Tüm Odaların Listelenmesi)

Bugün genel oda sayfasını geliştirdim. “Rooms.jsx” içinde, uygulamadaki tüm açık odaları listeleyen bir yapı kurdum. Burada kartlar halinde odalar ekrana geliyor. Her kartta oda başlığı, açıklaması, açan kişinin adı ve geri sayım yer alıyor. Eğer aktif oda yoksa kullanıcıya “Şu anda açık oda yok, yeni bir oda oluşturabilirsiniz” gibi bir bilgilendirme mesajı çıkıyor.

Bu sayfada ayrıca kapanmış odaların da listelenmesini sağladım. Kullanıcı isterse kapanmış odalara da bakabiliyor ama onlara yeni katkı yapılamıyor. Böylece uygulamada hem aktif hem de arşivlenmiş içerik bir arada görülebiliyor.

Rooms sayfası aslında uygulamanın sosyal tarafını ön plana çıkarıyor. Kullanıcı buradan istediği odaya katılabiliyor, katkı yapabiliyor ya da detayına giderek neler paylaşıldığını görebiliyor. Testlerde birden fazla oda açıp aralarındaki geçişi kontrol ettim. Hem aktif hem de kapanan odaların doğru şekilde listelendiğini gördüm.

Tarih ve İşyeri Amirinin İmzası	03/09/2025

GÜN 19

CreateRoom Sayfası (Yeni Oda Açma)

Bugün "CreateRoom.jsx" sayfasını geliştirdim. Burada kullanıcı yeni bir oda açabiliyor. Formda başlık, açıklama ve süre (örneğin 15 dk, 30 dk) seçilebiliyor. Ayrıca kullanıcı isterse oda için bir kapak görseli yükleyebiliyor. Resim seçilirse Storage'a yükleniyor, ilerleme çubuğunda yükleme yüzdesi gösteriliyor ve tamamlandığında resim linki Firestore'daki oda kaydına ekleniyor.

Süre seçimi kritik bir özellik oldu. Oda, belirlenen süre sonunda otomatik kapanıyor ve "aktif odalar" listesinden çıkıp "kapanan odalar"a taşınıyor. Bu da uygulamaya dinamik bir akış katıyor. Kullanıcıların sadece belirli zaman diliminde katkı yapabilmesi uygulamayı daha canlı hale getirdi.

Form doğrulamaları da yaptım. Başlık ve açıklama boş bırakılırsa uyarı mesajı çıkıyor. Başarılı şekilde oda açıldığında toast.success mesajı gösteriliyor ve kullanıcı otomatik olarak ilgili oda detay sayfasına yönlendiriliyor.

Bu sayfayı test ederken birkaç farklı süre seçtim ve odaların doğru zamanda kapanıp kapanmadığını kontrol ettim. Her şey sorunsuz çalıştı.

20: Yeni Oda Oluşturma (Create Room) Formu Arayüzü

Tarih ve İşyeri Amirinin İmzası	04/09/2025

GÜN 20

Genel Entegrasyon, ProtectedRoute ve Testler

Bugün proje genelini toparladım. “App.js” içinde **react-router-dom** ile tüm rotaları tanımladım: Home, Auth, MyPost, Rooms, RoomDetail, CreateRoom, AddEditBlog gibi sayfaları bir araya getirdim. Ayrıca giriş yapılmamış kullanıcıların özel sayfalara girmemesi için **ProtectedRoute** yapısını kurdum. Bu sayede kullanıcı giriş yapmadan MyPost veya CreateRoom sayfalarına girmeye çalışırsa otomatik olarak Auth ekranına yönlendiriliyor.

Navbar, giriş yapıldığında görünür hale geliyor; çıkış yapıldığında ise kullanıcı tekrar login ekranına dönüyor. Tüm akış boyunca ToastContainer aktif durumda, böylece kullanıcıya yapılan her işlemde bilgi veriliyor.

Son testleri yaptım: giriş → yeni oda açma → katkı yapma → blog oluşturma → kendi gönderilerimi görme → çıkış yapma. Tüm senaryoları tek tek denedim ve beklediğim gibi çalıştı. CSS tarafında ufak düzenlemelerle butonların ve kartların görünümünü daha düzenli hale getirdim.

Tarih ve İşyeri Amirinin İmzası	05/09/2025

STAJ DEFTERİNİN DOLDURULMASINDA VE DEĞERLENDİRİLMESİNDE DİKKAT EDİLECEK GENEL İLKELER

1. Staj defteri Bölüm tarafından aksi belirtilmediği sürece mürekkepli/tükenmez kalemle ya da antet yapısı korunarak bilgisayar çıktısı alınarak doldurulabilir.
2. Defterdeki bilgiler okunaklı yazılmalı, kullanılacak şekil, tablo veya fotoğraflardan defter sayfalarını aşmayanlar ilgili bölüme konulmalı; sayfa yazım alanını aşanlar uygun boyutta katlanıp ek olarak verilmelidir. Bu ekler metin içindeki değinme sırasına göre "EK.l, EK.2 ..." biçiminde numaralandırılmış olmalıdır.
3. Çalışma yapılan laboratuvar veya fabrikalara ilişkin yerleşim planları verilmelidir. Çalışılan makine, cihaz ve ölçüm aletlerinin özellikleri ile temel çalışma prensipleri belirtilmelidir.
4. Sadece kitap, broşür gibi basılı kaynaklardan aktarılan bilgi ve şekilleri içeren defterler değerlendirilmeyecektir. Basılı kaynaklardan alınmış bilgi ve belgelere (şekil ve fotoğraf gibi) mutlaka referans gösterilmelidir.
5. Aynı işyerinde staj yapan öğrenciler aynı bilgi ve kaynaklardan yararlanmış olabilir. Ancak bu durum, defterlerin birbirinin aynısı veya çok benzeri olmasını gerektirmez. Defterler biçim ve içerik bakımından özgün olmalıdır.
6. Belirtilen yerlerinde işyeri sorumlusunun onayı bulunmayan defterler değerlendirilemez.
7. Staj Yönergesinde belirtilen zorunlu nedenlerle iki farklı alandaki staj aynı dönemde yapılmışsa, her alan için ayrı defter doldurulmalıdır.
8. Uygulama bitiminde staj yapılmış olan kurum tarafından gönderilmesi gereken değerlendirme formunun bölüme ulaştırılmasından öğrenci sorumludur.
9. Öğrenci, yaptığı stajı jüri önünde sunacak; yapılan sunum ve staj belgeleri birlikte değerlendirilerek staj kabul edilecek veya edilmeyecektir.
10. Her Bölümün stajlarda dikkat edilmesi gereken konularla ilgili istedikleri diğer hususlar aşağıda maddeler halinde verilmiştir.

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

1. Öğrencilerin mezun olmadan önce her biri en az 20 iş günü olan iki stajı tamamlamaları gerekmektedir. 20 günlük staj bir defada tamamlanır.
2. Öğrenci staj yapmak istediği yeri Staj komisyonuna önerir ve komisyon onayını alır. Komisyon tarafından onaylanmamış veya geçerli mazereti olmaksızın stajını eksik bırakan öğrencinin stajı geçersiz sayılır.
3. Staj yapılan Kurum/Kuruluşta en az bir Bilgisayar Mühendisi olmalıdır.
4. Staj defteri, staj programına ve staj kurallarına uygun olarak, elle yazılacaktır. Her bir staj günü için en az bir sayfa olmak üzere toplamda en az 20 sayfa yazılmalıdır.
5. Staj süresince yapılan uygulama ve pratikler stajın yapıldığı kurumun çalışma alanından olacaktır.
6. Stajı süresince öğrencilerin staja devam edip etmedikleri staj komisyonunca ilgili kurumlar aranarak kontrol edilecektir. Staja devam etmeyen veya staj raporu uygun görülmeyen öğrencilerin stajları geçersiz sayılacaktır.
7. Öğrenciler aynı kurumda ve aynı zaman dilimleri arasında staj yapmış olsalar ve aynı projede çalışmış olsalar bile defterleri farklı olmak zorundadır. Aksi durumda ilgili öğrencilerin stajları geçersiz sayılır.
8. Staj yapılabilecek alanlar ve staj ile ilgili diğer detaylar bölüm staj komisyonunca bölümün web sayfasından ilan edilmektedir.

BİYOMÜHENDİSLİK BÖLÜMÜ

1. 20'şer iş gününden oluşan stajlardan ilki laboratuvar ikincisi ise işletme stajı olacaktır.
2. Laboratuvar stajında Biyomühendislik/Biyoloji bilimiyle ilgili alanlarda laboratuvar çalışma düzeni, güvenlik önlemleri, kullanılan cihazlar ve analiz yöntemleri incelenip genel kurallar ışığında rapor halinde sunulacaktır.
3. İşletme stajında ise prosesin akım şeması, yönetim organizasyonu, kütle ve enerji denklikleri oluşturulacak, işgücü analiz edilecektir. Proseste önerilecek iyileştirmeler öneri olarak sunulacaktır.

ÇEVRE MÜHENDİSLİĞİ BÖLÜMÜ

1. Stajlar laboratuvar/büro ve şantiye/işletme olmak üzere iki aşamalı olup ilk aşamada Laboratuvar/Büro stajı ikinci aşamada ise şantiye/işletme stajı yapılmalıdır.
2. Stajların yapılabileceği temel alanlar: Çevre kimyası, çevre mikrobiyolojisi, çevre biyoteknolojisi ile ilgili laboratuvar çalışmaları, su kirlenmesi ve kontrolü, su temini ve atıksuların uzaklaştırılması, su ve atıksu arıtma teknolojileri, hava kirlenmesi ve kontrolü, katı ve tehlikeli atıkların yönetimi ve bertarafı, gürültü kirliliği kontrolü, endüstriyel atıkların yönetimi, çevresel etki değerlendirmesi, çevre yönetimi ve planlaması.
3. İki dönemlik zorunlu stajın her biri 20' şer iş gününden oluşmalı ve stajlar aynı temel alanları içeren kurumlarda yapılmamalıdır.
4. Staj raporu, yazım kurallarına uygun bir şekilde mürekkepli kalemle hazırlanmalıdır.
5. Staj ile ilgili işletmenin akım şeması çizilmeli ve atık oluşturan birimler, atık miktarları ve atık yükleri ayrıntılı bir şekilde verilmelidir.
6. Yapılan deneylerin ve kullanılan cihazların özellikleri, prensipleri, deneyleri amaçları ve analiz sonuçlarının yorumlan mutlaka verilerek standartlarla karşılaştırılmalıdır.
7. Staj süresince yapılan işler ve/veya elde edilen veriler çevre mevzuatı ile ilişkilendirilip yorumlanmalıdır.

ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

1. İlk staja 2.sınıftan en az 40 AKTS, ikinci staja da 3. sınıftan en az 30 AKTS ders alan öğrenciler başvurabilir.
2. İki staj ayrı iş yerlerinde yapılmalıdır. Özel durumlar Staj Komisyonunun iznine tabiidir. Her stajda, işyerinin farklı birimlerinde bulunulmaya çalışılmalıdır.
3. Staj yapılacak işyerinde, en az 1 (bir) Elektrik/Elektronik/Elektrik-Elektronik Mühendisi olmalıdır.
4. a. Staj defteri mürekkepli ya da tükenmez kalemle günlük iş planına göre düzgün bir şekilde elle yazılmalı; gerekiyorsa fotoğraflar, çizimler ve projeler defterin ekinde verilmelidir.
   b. Staj defteri günlük yazılmalı, yazılanlar işyeri ile uyumlu olmalıdır. Defter resimlerle ya da şekillerle doldurulmamalıdır.
   c. Defterde ilgili yerlerde sorumlu mühendis bilgileri açık olmalı, kurum kaşe ve mührü mutlaka vurulmalıdır. Her sayfada, sorumlu kişinin imzası olmalıdır.
   d. Staj defterlerinin başında staj yerinin organizasyonu ve faaliyetleri hakkında bilgi verilmelidir (en fazla 3 sayfa olabilir).
   e. Staj defterleri birbirinin aynı olan öğrencilerin stajları kabul edilmeyecektir.
5. Staj sonunda, staj değerlendirme formu kurum tarafından doldurulduktan sonra posta ile bölüme gönderilebilir ya da gizli kaşesi taşıyan kapalı ve mühürlü zarf içinde öğrenciye teslim edilebilir. Değerlendirme formu kapalı zarf içinde öğrenciye teslim edilmişse, öğrenci bölüme teslim eder.
6. Defter ve (öğrenciye teslim edilmişse) staj değerlendirme formu öğretim yılının ilk 15 günü içerisinde (staj dersleri 1 ve 2 alınmasa dahi) bölüme imza karşılığında teslim edilmelidir. Teslim etmeyen öğrencilerin stajları kabul edilmeyecektir. Defterler öğrenciye geri verilmeyeceğinden dolayı, öğrenci teslim edilmeden önce sunum hazırlıkları için gerekli dokümanları almalıdır.
7. Öğrenciler, öğretim yılının ilk 15 günü içerisinde staj değerlendirme formunun bölüme ulaşıp ulaşmadığını öğrenmelidir. Eğer ulaşmamış ise staj yaptıkları kurumla iletişime geçerek, formların bölüme ulaşmasını sağlamalıdır.
8. Staj hariç tüm derslerinden başarılı olanlar ve Sınav haftalarını kapsamayacak tarihlerde olmak şartıyla, bulunduğu dönemdeki derslerin hiç birisinden devam mecburiyeti olmayan öğrenciler Öğretim dönemi içerisinde staj yapabilir.
9. İlk defa staj yapacak olan 2. sınıf öğrencileri önceden en az bir staj sunumu dinlemelidir. Aksi halde, staj evrakları onaylanmayacaktır.

İNŞAAT MÜHENDİSLİĞİ BÖLÜMÜ

Staj şantiye ve büro stajı olmak üzere iki alanda yapılacaktır.

Şantiye Stajı;

Yapı, yol, su yapıları vb. şantiyelerden birisinde proje uygulaması, çeşitli imalatların yapılması veya denetleme konularını kapsar.

Büro Stajı;

Fiyat Analizleri keşif özetleri, İhale, Çeşitli projelerin düzenlenmesi (mimari statik, yol, su getirme ve kanalizasyon vb. Dinamik Hidrolik hesaplamaların yapılması, İnşaat Mühendisliği ile ilgili paket programları kapsar.

JEOLOJİ MÜHENDİSLİĞİ BÖLÜMÜ

Staj, jeoloji mühendisliğiyle ilgili arazi veya laboratuvar çalışması gerçekleştirilen kurum veya kuruluşlarda yapılmalıdır.

KİMYA MÜHENDİSLİĞİ BÖLÜMÜ

1. İşletmenin her bölümü; hammadde, ürün, reaksiyon şartları ve kapasite yönünden dikkatle incelenmelidir.
2. Her bölüm için ayrı ayrı ve tüm proses için enerji ve madde balansı yapılmalıdır.
3. İşletmenin her bölümünün akış şemaları ve bu bölümler arasındaki ilişkiyi gösteren bir total akış şeması çizilmelidir.
4. Tesiste bulunan önemli cihazların özellikleri, yapıları ve çalışma prensipleri incelenmelidir.
5. İmalat ve kalite kontrol için yapılan analizler hakkında bilgiler derlenmelidir.
6. İşletmede kullanılan yardımcı tesisler ve bunların işletme ekonomisi, çevresel duyarlılık ve teknolojik gelişme yönlerinden katkılarını da belirten bilgiler derlenmelidir.
7. İşletmenin organizasyon şeması oluşturulmalıdır.
8. İşletmede elde edilen ürünler için maliyet analizlerine ilişkin bilgiler derlenmelidir.
9. İşletmede kullanılan üretim teknolojilerinin modern teknolojilerle karşılaştırılması yapılarak verimliliğin ve kapasitenin arttırılması ve maliyetlerin düşürülmesi için neler yapılabileceğini belirten bir değerlendirme yapılmalıdır.

MAKİNE MÜHENDİSLİĞİ BÖLÜMÜ

Stajlar aşağıda belirtilen konuları kapsayan iki farklı alanda yapılmalıdır.

I. Alan: Atölye

Atölye alanındaki çalışmaların 10 iş gününü talaşlı imalat, 5 iş gününü döküm, 5 iş gününü de kaynak ve şekillendirme işlemleri oluşturur.

Atölye çalışması; imalat yöntemleri, imalattaki iş sırası ve imalat makinalarının tanımı, belirgin özellikleri ve çalışma sistemlerini kapsar. Teknolojik bilgi, gözlem, imalat resimlerinin çizimleri ve uygulamaya dayanır. Bu çalışmalar mümkün olduğunca, seri üretim yapılan ve tam teşekküllü atölyeleri bulunan kurumlarda yapılır.

Talaşlı İmalat Yöntemleri: Bu bölümde torna, taşlama, freze, matkap, planya gibi tezgâhlarda talaş kaldırma işlemleri bilfiil takip edilmelidir. Bu işlemler sırasında kullanılan her türlü alet ve tezgâhların özellikleri araştırılıp tezgâh üzerinde uygulama yapılacak, tezgâhlarda imal edilen parçaların teknik resimleri norm ve standartlara uygun şekilde kurşun kalemle deftere çizilerek, parçaların tezgâha bağlama ve işleme yöntemleri kısaca açıklanacaktır. Ayrıca varsa bilgisayar destekli tezgâhlarla ilgili program hazırlanması, ofset işlemleri ve tezgâhlarda parçaların işlenmesi takip edilerek gerekli açıklamalar yazılacaktır.

Döküm: Dökümcülük, Döküm Kalıpçılığı ve Maden Ergitme Tekniği olarak iki grup altında toplanabilir. Staj sırasında, genel döküm bilgileri ışığında dökümcülükte kullanılan ocaklar, kapasite ve verimleri, dökümcülük alet ve gereçleri, kalıplama yöntemleri, kalıba ergiyik metalin dökülmesi, döküm sonrası işlemlerin değerlendirilmesi ve imalat resimlerinin çizilmesi, döküm çeşitleri ve döküm işleminde dikkat edilecek hususlar incelenip deftere yazılmalıdır.

Kaynak ve Plastik Şekil Verme: Kaynak yöntemleri hakkında bilgi verilip yapılan uygulamalara ait Teknik Resimler çizilip gerekli açıklamalar yapılmalıdır. Plastik şekil verme, dövme, haddeleme ve saç işleme gibi işlemlerin özellikleri incelenmeli ve gerekli açıklamalar yapılmalıdır.

II. Alan: Fabrika Organizasyonu ve Yönetimi

Fabrika Organizasyonu ve Yönetimi alanındaki stajlar, ürün ve/veya hizmet üreten işletmelerde yapılabileceği gibi ısıtma, soğutma ve havalandırma projelerini yapan işletmelerde de yapılabilir. 20 iş gününü kapsayan uygulama aşağıdaki konularda olmalıdır.

Fabrika Organizasyonu ve Yönetimi (süresi 2 hafta): Fabrikanın örgütsel yapısı, fabrikada yer alan iş etüdü çalışmalarının araştırılması, üretim planlama ve kontrol teknikleri, iş güvenliği, işçi-işveren ilişkileri, satın alma işlemlerinin uygulanış şekli, hammadde temini, depolama ve stoklamanın işletme içindeki önemi, stok bulundurma nedenleri, stok kontrolde maliyet unsurları, bakım üniteleri ve hedefleri, üretimi artırma çabaları, kalite kontrol düzenleri, toplam kalite yönetimine ilişkin çalışmaların tespiti, güç ve enerji ünitelerinin analizi (elektrik dağıtım şebekesi bağlantı ve güçleri ile), AR-GE faaliyetlerinin araştırılması.

Üretim ve Montaj İşlemleri (süresi 2 hafta): Üretimi yapılan malzeme ve teçhizatın projelendirme aşamalarının etüdü; üretimde kullanılan tezgâh ve makinelerde iş akışı ve imalat zamanının incelenmesi; montajda uygulanan yöntem ve teknikler belirlenerek varsa önerilerle birlikte deftere yazılır.

MEKATRONİK MÜHENDİSLİĞİ BÖLÜMÜ

Her biri 20 iş gününden ibaret, staj-1 ve staj-2 olarak belirtilen stajlar farklı kurumlarda yapılmalıdır. Ancak büyük ölçekli kurumların farklı yerleşkelerde yer alan, farklı birimlerinde(AR-GE, Üretim, Tasarım, Kalite vb. birimlerinde) öğrenciler her iki stajını da yapabilirler.

1. Öğrenciler stajlarını, Mekatronik Mühendisinin bulunduğu birimlerde yapmalıdırlar. Şayet Mekatronik Mühendisi bulunmuyorsa, Makine Mühendisi veya Elektrik Elektronik Mühendisinin bulunma şartı aranır.
2. Staj defterleri, defter sayfalarındaki formatın korunması şartı ile bilgisayar çıktısı şeklinde yazılarak hazırlanabilir. Zorunlu kalınması halinde el yazısı ile staj defteri yazılabilir.
3. Staj defterinde stajın yapıldığı her bir günün tarihi açıkça belirtilmelidir, her bir sayfası staj yapılan ilgili birimin mühendisi tarafından imzalanıp kaşelenmelidir. Kaşede unvan ve diploma numarası belirtilmiş olmalıdır.
4. Staj yapacak öğrenciler, staj defterlerinde, staj yaptıkları kurumu tanıtan bilgilerini, ilgili birimin faaliyet alanlarını, ürün ile ilgili detaylı teknik bilgileri, öğrencinin bizzat kendisi tarafından gerçekleştirmiş olduğu en az 5 farklı uygulamaları kapsayacak biçimde, açıklayıcı bir dilde yazarak aktarmalıdırlar. Staj defterine aktarılan bilgiler gerek görüldüğünde teknik çizimlerle, resimlerle ve açıklayıcı şemalarla desteklemelidirler.
5. Staj yapacak öğrenciler uygulamalarını mekanik ve elektrik-elektronik sistemlerini barındıran, yazılım içerikli ürünler üzerine gerçekleştireceklerdir.

METALURJİ VE MALZEME MÜHENDİSLİĞİ BÖLÜMÜ

Staj raporu, staj defterindeki biçim korunmak şartıyla ve 20 sayfadan az olmamak kaydıyla okunaklı ve tertipli bir şekilde el yazısıyla veya bilgisayar çıktısı olarak hazırlanabilir.

Stajlar için demir-çelik fabrikaları, döküm fabrikaları, otomotiv ve makine imalat sanayi, alüminyum ve bakır gibi demir dışı metal üretim fabrikaları, şişe -cam sanayi ve ısıl işlem fabrikaları gibi metalürji ve malzeme konusunda faaliyet gösteren fabrikalar seçilebilir.

Stajlar aşağıda belirtilen konuları kapsayan iki farklı alanda yapılmalıdır.

I. Alan: Üretim

a) Öğrencilerin, Metalurji ve Malzeme Mühendisliği kapsamı içerisine giren üretim stajı yapmaları gerekmektedir.

b) Öğrenciler yukarda anılan sektörlerde 20 iş günü staj yapmakla sorumlu olup, bu sektörlerde yaptıkları staj süresince üretime bilfiil katılmak şartı aranır.

II. Alan: Fabrika Organizasyonu ve Yönetimi

a) Fabrikanın organizasyonu, fabrikada yer alan iş etüdü ve çalışmaları araştırılması, üretim planlama ve kontrol teknikleri, iş güvenliği, işçi-işveren ilişkileri, satın alma işlemlerinin uygulanışı, hammadde temini, depolama ve stoklamanın işletme içindeki önemi, stok bulundurma nedenleri, stok kontrolde maliyet unsurları, bakım üniteleri ve hedefleri, üretimi artırma çabaları, kalite kontrol düzenleri, toplam kalite yönetimine ilişkin çalışmaların tespiti, güç ve enerji ünitelerinin analizi (elektrik dağıtım şebekesi bağlantı ve güçleri ile), araştırma-geliştirme (AR-GE) faaliyetlerinin araştırılması incelenecektir.

b) Fabrika Organizasyonu ve Yönetim Stajı 20 iş günüdür.

YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

1. Staj raporunda uygulamalar ağırlıklı olarak yer alacak ve gerekli yerlerde teorik bilgi verilecektir.
2. Eğer günlük iş yerine proje üzerinde çalışma yapılmışsa, staj raporunda günlük anlatım yerine proje ve nasıl gerçekleştirildiği açıklanmalıdır.
3. Staj raporu bilgisayar çıktısı olarak hazırlanacaktır.
4. Staj yapılan yerde mutlaka yazılım mühendisi bulunacaktır. Bu mümkün değilse, bilgisayar mühendisi bulunacaktır.
5. Staj raporunun yazılmasında F.Ü. Fen Bilimleri Enstitüsü yüksek lisans tez yazım kuralları esas alınacaktır.
6. Aksi ilan edilmedikçe staj sunumları, Güz döneminin 6. haftasının Çarşamba günü ilgili jüri huzurunda yapılacaktır.
7. Staj belgelerinin benzerlik oranı test edilecek ve sonuç rapor sunulacaktır.